임베디드 시스템

Embedded Systems

Bluetooth
Android

강원대학교 컴퓨터공학과 권용훈

블루투스(1/4)

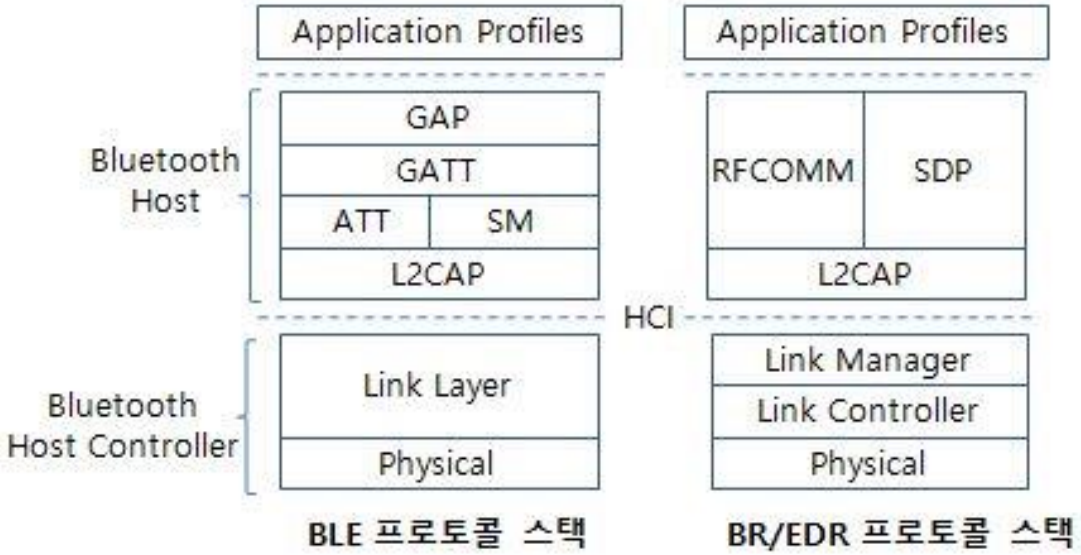
블루투스(Bluetooth)

- 근거리 무선 통신(PAN)의 기술 중 하나이며, IEEE 802.15.1로 등록되어 널리 사용되고 있다.
- 1994년 에릭센을 시작으로 인텔, 노키아 등의 회사가 참여하여 만들어진 표준 기술이다.
- 블루투스는 2.45GHz 대역을 사용하여 와이파이(Wi-Fi)와 충돌할 가능성이 있다.
- 제공되는 프로파일(profile)에 따라 사용할 수 있는 용도가 달라진다.
- A2DP, AVRCP, HSP, HFP, PBAP, HID, PAN, SPP 등의 프로파일이 있다.
- Raspberry Pi 3 B+ Model에는 4.2/BLE 버전이 탑재되어 있다.
    - Bluetooth Classic: 대용량의 텍스트와 멀티미디어를 처리가능한 일반적인 버전
    - Bluetooth Low Energy(BLE): 저전력을 사용하며 크기가 작은 데이터를 처리하는 버전
    - Bluetooth Smart: Classic과 Low Energy를 모두 사용하는 버전 (일반적인 형태)

블루투스(2/4)

블루투스 프로토콜 스택

- 블루투스 기기 사이에서 데이터를 송수신하기 위해 지정한 규약
- HCI를 기준으로 호스트 컨트롤러(Host controller) 프로토콜과 호스트(Host) 프로토콜로 구분
- 호스트 컨트롤러 – 아래쪽에 위치하며 블루투스 모듈에 해당
- 호스트 – 블루투스 모듈을 제어하고 어플리케이션을 수행
- 블루투스의 프로토콜은 프로파일에 따라 상이하다.



블루투스(3/4)

주요 블루투스 프로토콜

블루투스 프로토콜		설명
BNEP	Bluetooth Network Encapsulation Protocol	네트워크 연결 프로토콜
HIDP	Human Interface Device Protocol	마우스 등 HID 장치 프로토콜
RFCOMM	RF Communication Protocol	간단한 RF 통신 프로토콜
L2CAP	Logical Link Control and Adaptation Protocol	논리 링크 제어 프로토콜
SCO	Synchronous Connection Oriented Protocol	동기 연결 프로토콜
OBEX	Object Exchange Protocol	파일 전송 같은 객체 교환 프로토콜
ATT	LE Attribute Protocol	LE 속성 데이터 전송 프로토콜
SDP	Service Discovery Protocol	서비스 검색 프로토콜

블루투스(4/4)

Bluez software

- 커널 모듈을 포함한 블루투스 계층과 프로토콜, 사용자 라이브러리, 설정, 테스트 도구 등을 제공하는 패키지

약자	설명
bluetootd	블루투스 서비스 데몬 프로그램, 모든 블루투스 프로파일/서비스 구동
obexd	OBEX FTP 서비스 데몬 프로그램
bluetoothctl	bluetoothtd와 통신하며 장치 검색, 연결 등 기본 동작 실행
hciconfig	블루투스 HCI장치를 설정하는 프로그램
hcitool	블루투스 HCI장치와 연결하고 명령 전송 제어 프로그램
hcidump	블루투스 HCI장치와 주고받은 메시지를 덤프하는 프로그램
hciattach	블루투스 HCI장치를 dev로 붙이거나 떼는 프로그램
sdptool	블루투스 서비스를 검색하고 출력하는 프로그램
rfcomm	커널의 블루투스 시스템과 통신하며 RFCOMM 설정 및 처리하는 프로그램

블루투스 환경 설정(1/3)

블루투스 통신 SDP 프로토콜 설정

```
sudo nano /etc/systemd/system/dbus-org.bluez.service
```

- 해당 위치에 '—compat' 추가

```
[Unit]
Description=Bluetooth service
Documentation=man:bluetoothd(8)
ConditionPathIsDirectory=/sys/class/bluetooth

[Service]
Type=dbus
BusName=org.bluez
ExecStart=/usr/lib/bluetooth/bluetoothd --compat
NotifyAccess=main
#WatchdogSec=10
#Restart=on-failure
CapabilityBoundingSet=CAP_NET_ADMIN CAP_NET_BIND_SERVICE
LimitNPROC=1
ProtectHome=true
ProtectSystem=full

[Install]
WantedBy=bluetooth.target
Alias=dbus-org.bluez.service
```

- 변경된 설정 적용

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart bluetooth
```

```
sudo chmod 777 /var/run/sdp
```

블루투스 환경 설정(2/3)

SPP 서비스 등록

- SDP에 등록된 서비스 리스트 확인

```
sudo sdptool browse local
```

- SPP & RFCOMM 서비스 설정

```
sudo sdptool add --channel=4 SP
```

```
pi@raspberrypi:~ $ sudo sdptool add --channel=4 SP
Serial Port service registered
```

- SPP & RFCOMM 서비스 가능 여부 확인

```
sudo sdptool browse local
```

- 4번 채널이 설정되어 있다면 생략

```
Service Name: Serial Port
Service Description: COM Port
Service Provider: BlueZ
Service RecHandle: 0x1000a
Service Class ID List:
  "Serial Port" (0x1101)
Protocol Descriptor List:
  "L2CAP" (0x0100)
  "RFCOMM" (0x0003)
    Channel: 4
Language Base Attr List:
  code_ISO639: 0x656e
  encoding:    0x6a
  base_offset: 0x100
Profile Descriptor List:
  "Serial Port" (0x1101)
    Version: 0x0100
```

블루투스 환경 설정(3/3)

블루투스 기기 검색 기능 활성화

- HCI device에 대한 정보 확인 및 설정

```
sudo hciconfig hci0
```

```
pi@raspberrypi:~ $ sudo hciconfig hci0
hci0:   Type: Primary  Bus: UART
        BD Address: B8:27:EB:84:38:96  ACL MTU: 1021:8  SCO MTU: 64:1
        UP RUNNING
        RX bytes:813 acl:0 sco:0 events:53 errors:0
        TX bytes:2524 acl:0 sco:0 commands:53 errors:0
```

```
sudo hciconfig hci0 piscan
```

```
hci0:   Type: Primary  Bus: UART
        BD Address: B8:27:EB:84:38:96  ACL MTU: 1021:8  SCO MTU: 64:1
        UP RUNNING PSCAN ISCAN
        RX bytes:827 acl:0 sco:0 events:55 errors:0
        TX bytes:2565 acl:0 sco:0 commands:55 errors:0
```

- ISCAN: 상대 블루투스 기기에서 검색 가능
- PSCAN: 상대 블루투스 검색 가능

갤럭시 탭 페어링(1/4)

블루투스 기기 페어링 설정

- 블루투스 관리자 실행 및 블루투스 모듈 정보 확인

bluetoothctl

```
pi@raspberrypi:~ $ bluetoothctl
[NEW] Controller B8:27:EB:84:38:96 raspberrypi [default]
[bluetooth]#
```

show

```
[bluetooth]# show
Controller B8:27:EB:84:38:96
        Name: raspberrypi
        Alias: raspberrypi
        Class: 0x480000
        Powered: yes
        Discoverable: yes
        Pairable: yes
        UUID: Headset AG                (00001112-0000-1000-8000-00805f9b34fb)
        UUID: Generic Attribute Profile (00001801-0000-1000-8000-00805f9b34fb)
        UUID: A/V Remote Control        (0000110e-0000-1000-8000-00805f9b34fb)
        UUID: Generic Access Profile    (00001800-0000-1000-8000-00805f9b34fb)
        UUID: PnP Information           (00001200-0000-1000-8000-00805f9b34fb)
        UUID: A/V Remote Control Target (0000110c-0000-1000-8000-00805f9b34fb)
        UUID: Audio Source              (0000110a-0000-1000-8000-00805f9b34fb)
        UUID: Handsfree Audio Gateway   (0000111f-0000-1000-8000-00805f9b34fb)
        Modalias: usb:v1D6Bp0246d052B
        Discovering: no
```

- 블루투스 모듈 전원 켜기

power on

```
[bluetooth]# power on
Changing power on succeeded
```

- agent 설정 및 활성화

agent on

default-agent

```
[bluetooth]# agent on
Agent registered
[bluetooth]# default-agent
Default agent request successful
```

갤럭시 탭 페어링(2/4)

블루투스 기기 페어링

- 갤럭시 탭 MAC Address 확인

[설정] -> [디바이스 정보] -> [상태]-> 블루투스 주소

- 라즈베리 파이 블루투스 기기 탐색

scan on

- 갤럭시 탭의 MAC 주소가 나올 때까지 수행

```
[CHG] Device 73:EE:A7:5B:F4:A9 RSSI: -47
[CHG] Device 6C:21:76:A1:F4:F4 RSSI: -77
[CHG] Device D4:AE:05:82:D1:72 RSSI: -45
[CHG] Device 77:AD:81:29:34:71 RSSI: -55
[NEW] Device 04:3F:91:99:7E:80 04-3F-91-99-7E-80
[NEW] Device 74:75:45:F2:0E:25 74-75-45-F2-0E-25
[CHG] Device 73:D4:65:DD:EF:AA RSSI: -59
[CHG] Device 4C:1C:36:40:1B:9A RSSI: -53
[CHG] Device B0:99:D7:65:55:0D RSSI: -81
[CHG] Device 57:1D:13:C0:CF:2C RSSI: -59
[CHG] Device 73:EE:A7:5B:F4:A9 RSSI: -72
```

- 탐색 종료

scan off

갤럭시 탭 페어링(3/4)

블루투스 기기 페어링

- 갤럭시 탭 정보 확인

```
info [MAC Address]
```

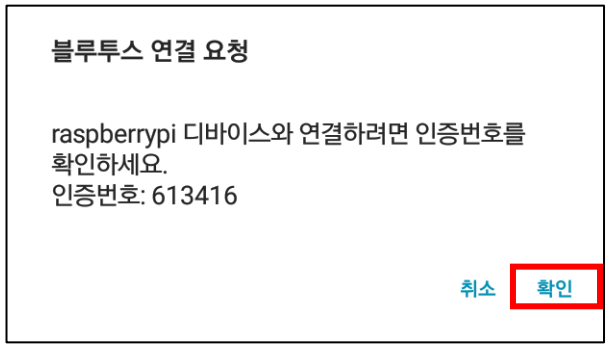
```
[bluetooth]# info D4:AE:05:82:D1:72
Device D4:AE:05:82:D1:72
        Name: Galaxy Tab A (2016)
        Alias: Galaxy Tab A (2016)
        Class: 0x5a020c
        Icon: phone
        Paired: no
        Trusted: no
        Blocked: no
        Connected: no
        LegacyPairing: no
```

- 페어링(라즈베리 파이 -> 갤럭시 탭)

```
pair [MAC Address]
```

- 상호 인증

```
[bluetooth]# pair D4:AE:05:82:D1:72
Attempting to pair with D4:AE:05:82:D1:72
[CHG] Device D4:AE:05:82:D1:72 Connected: yes
Request confirmation
[agent] Confirm passkey 613416 (yes/no): yes
[CHG] Device D4:AE:05:82:D1:72 Modalias: bluetooth:v0075p0100d0200
[CHG] Device D4:AE:05:82:D1:72 UUIDs: 00001105-0000-1000-8000-00805f9b34fb
[CHG] Device D4:AE:05:82:D1:72 UUIDs: 0000110a-0000-1000-8000-00805f9b34fb
[CHG] Device D4:AE:05:82:D1:72 UUIDs: 0000110c-0000-1000-8000-00805f9b34fb
[CHG] Device D4:AE:05:82:D1:72 UUIDs: 00001112-0000-1000-8000-00805f9b34fb
[CHG] Device D4:AE:05:82:D1:72 UUIDs: 00001115-0000-1000-8000-00805f9b34fb
[CHG] Device D4:AE:05:82:D1:72 UUIDs: 0000111f-0000-1000-8000-00805f9b34fb
[CHG] Device D4:AE:05:82:D1:72 UUIDs: 00001200-0000-1000-8000-00805f9b34fb
[CHG] Device D4:AE:05:82:D1:72 UUIDs: 00001800-0000-1000-8000-00805f9b34fb
[CHG] Device D4:AE:05:82:D1:72 UUIDs: 00001801-0000-1000-8000-00805f9b34fb
[CHG] Device D4:AE:05:82:D1:72 ServicesResolved: yes
[CHG] Device D4:AE:05:82:D1:72 Paired: yes
Pairing successful
```



갤럭시 탭 페어링(4/4)

블루투스 기기 페어링

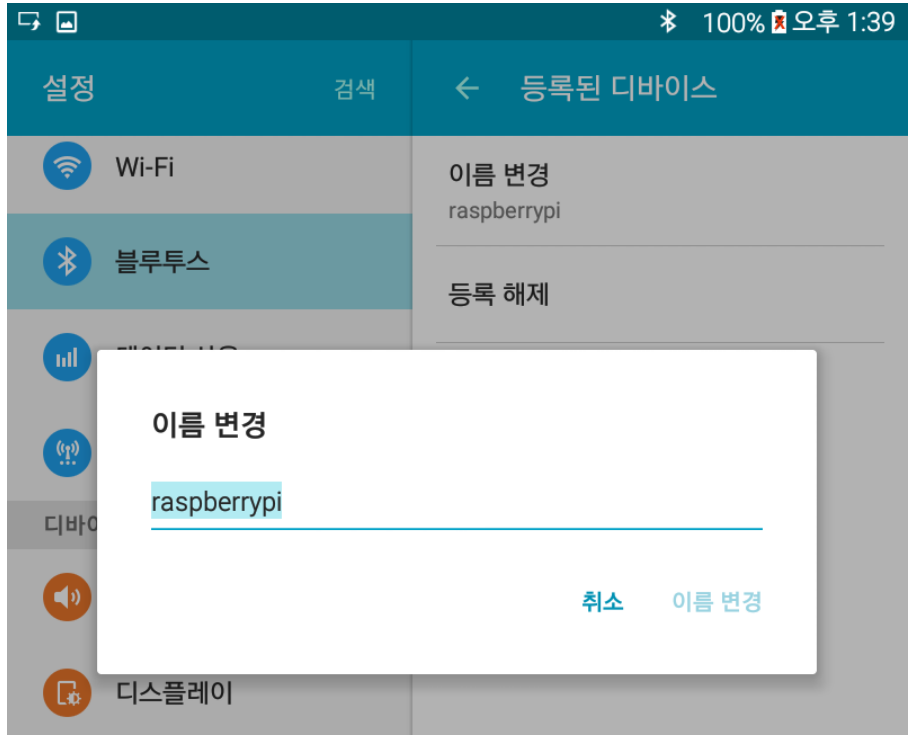
- 신뢰하는 기기 등록

trust [MAC Address]

```
[bluetooth]# trust D4:AE:05:82:D1:72
[CHG] Device D4:AE:05:82:D1:72 Trusted: yes
Changing D4:AE:05:82:D1:72 trust succeeded
```

- 분반 간의 라즈베리 파이 구분

등록된 기기의 우측의 설정으로 이름 변경 [학번 이름]



블루투스 프로그래밍(1/5)

wiringSerial 라이브러리

int serialOpen (const char *device, const int baud)		
위 치	wiringSerial.h	
기 능	지정된 디바이스와 Serial 통신 초기화 및 통신 시작	
매개변수	char *device	디바이스
	int baud	통신 속도
리턴 값	int	-1 : 에러발생 / 통신할 디바이스 정보

void serialClose (const int fd)		
위 치	wiringSerial.h	
기 능	시리얼 통신 종료	
매개변수	int fd	통신 중인 디바이스
리턴 값	void	없음

void serialFlush (const int fd)		
위 치	wiringSerial.h	
기 능	송수신 데이터를 모두 버림	
매개변수	int fd	통신할 디바이스
리턴 값	void	없음

void serialPutchar (const int fd, const unsigned char c)		
위 치	wiringSerial.h	
기 능	연결된 디바이스에 1byte 데이터 전송	
매개변수	int fd	통신 중인 디바이스
	unsigned char c	전송하는 byte 데이터
리턴 값	void	없음

void serialPuts (const int fd, char *s)		
위 치	wiringSerial.h	
기 능	연결된 디바이스에 1byte 데이터 전송	
매개변수	int fd	통신 중인 디바이스
	char *s	전송되는 문자열
리턴 값	void	없음

void serialPrintf (int fd, char *message, ...)		
위 치	wiringSerial.h	
기 능	연결된 디바이스에 데이터 전송	
매개변수	int fd	통신 중인 디바이스
	char *message	전송되는 문자열
리턴 값	void	없음

int serialDataAvail (int fd) ;		
위 치	wiringSerial.h	
기 능	읽을 수 있는 데이터 수 조사	
매개변수	int fd	통신 중인 디바이스
리턴 값	int	읽을 수 있는 데이터 수 / 없을 경우 -1

int serialGetchar (int fd) ;		
위 치	wiringSerial.h	
기 능	데이터 읽어오기	
매개변수	int fd	통신 중인 디바이스
리턴 값	int	읽은 데이터 / 없을 경우 -1

블루투스 프로그래밍(2/5)

프로토콜(Protocol)

- 서로 다른 장비 사이에 데이터를 송수신하기 위한 규칙을 정하는 것
- 일반적으로 다음과 같은 규칙을 갖는다.

시작 문자	제어센서 이름	구분자	센서 제어값	종료 문자

- 실습용 예제는 다음과 같은 프로토콜을 채택

시작문자	구분자	종료문자
@	,(콤마)	#

[AI Solution Kit ==> App]			
커맨드 (제어 센서 이름)	데이터 (센서 제어 값)	동작	예시
TMP	두 자리 숫자	온도 값 전달	@TMP,17#
HUM	두 자리 숫자	습도 값 전달	@HUM,45#
IMG	JPG이미지 데이터 마지막 3바이트 ###	이미지 데이터 전달	@IMG,..####

[App ===> AI Solution Kit]			
커맨드 (제어 센서 이름)	데이터 (센서 제어 값)	동작	예시
RED	한 자리 숫자 0 – OFF 1 – ON	Red LED On/Off	@RED,0# @RED,1#
YED		Yellow LED On/Off	@YED,0# @YED,1#
GED		Green LED On/Off	@GED,0# @GED,1#
LED		LED On/Off	@LED,0# @LED,1#
SHT		카메라 촬영	@SHT#

블루투스 프로그래밍(3/5)

데이터 송수신 예제 수행

- 예제 파일 중 [EX_BTServer.c] 를 컴파일

```
gcc -o BTServer BTServer.c -lwiringPi
```

- 실행 후 송수신 대기

```
sudo sdptool add --channel=4 SP    //시스템 실행 시 1회
```

```
sudo rfcomm watch /dev/rfcomm0 4 ./BTServer
```

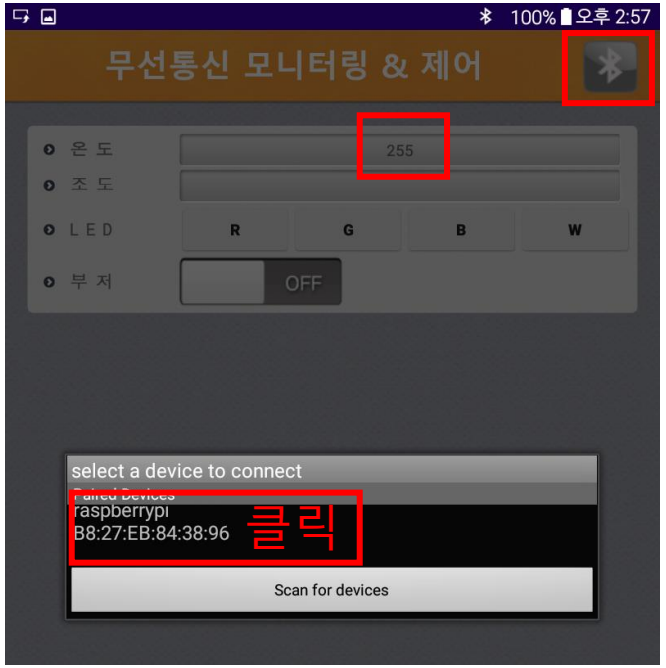
```
pi@raspberrypi:~ $ sudo rfcomm watch /dev/rfcomm0 3 ./test
Waiting for connection on channel 3
```

- 어플리케이션 설치 및 실행

e-루리 상 apk 파일 다운 -> 유선으로 전송 및 설치

중앙의 [블루투스 예제] 실행 -> 우측 상단의 [버튼] 클릭

```
pi@raspberrypi:~ $ sudo rfcomm watch /dev/rfcomm0 3 ./test
Waiting for connection on channel 3
Connection from D4:AE:05:82:D1:72 to /dev/rfcomm0
Press CTRL-C for hangup
Bluetooth init
Send Sensor Data
Send Sensor Data
Send Sensor Data
Send Sensor Data
Send Sensor Data
Send Sensor Data
Send Sensor Data
Send Sensor Data
Send Sensor Data
```



블루투스 프로그래밍(4/5)

블루투스 통신 송신 예제 #1

- gcc -o [실행파일] [.c 파일] -lwiringPi
- 실행: sudo rfcomm watch /dev/rfcomm0 4 ./실행파일

```
#include <stdio.h>
#include <wiringPi.h>
#include <wiringSerial.h>

void  hw_init();
void  initBT();
int   g_hBT;                //- BT통신 객체


int main(){

    unsigned int nNextTime;
    initBT();
    hw_init();
```

```
    while(1){
    // 1초 간격으로 가상의 센서 데이터 전송
          if(millis() > nNextTime + 1000){
            printf("Send Sensor Data\n");
            serialPrintf(g_hBT, "@TMP,32#");
            nNextTime = millis();
          }
    }
    return 0 ;
}

void hw_init(void)
{
    wiringPiSetup();
}

void initBT()
{
  g_hBT = serialOpen("/dev/rfcomm0", 115200);
  printf("Bluetooth init\n");
}
```

블루투스 프로그래밍(5/5)

블루투스 통신 수신 예제 #2

- 컴파일 및 실행 동일

```c
#include <stdio.h>
#include <string.h>
#include <wiringPi.h>
#include <wiringSerial.h>

char  rx_buf[BUFSIZ] = "";     //- 수신 데이터 저장 변수
int   recv_size;               //- 수신 데이터 사이즈
int   g_hBT;                  //- BT통신 객체

void hw_init(void)
{
    wiringPiSetup();
}
void initBT()
{
    g_hBT = serialOpen("/dev/rfcomm0", 115200);
    printf("Bluetooth init\n");
}
```

```c
int main(){
    unsigned int nNextTime;
    initBT();
    hw_init();
    while(1){
        // 0.25초 간격으로 수신 데이터 체크

        if(millis() > nNextTime + 250){
                int i=0;

                memset(rx_buf, 0, sizeof(rx_buf));
                while(serialDataAvail(g_hBT) > 0)
                {
                    rx_buf[i] = serialGetchar(g_hBT);
                    if(rx_buf[i] == '#')
                    {
                        if(rx_buf[0] == '@')
                            printf("RX Data = %s\n", rx_buf);
                        break;
                    }
                    i++;
                }
            nNextTime = millis();
        }
    }
    return 0 ;
}
```

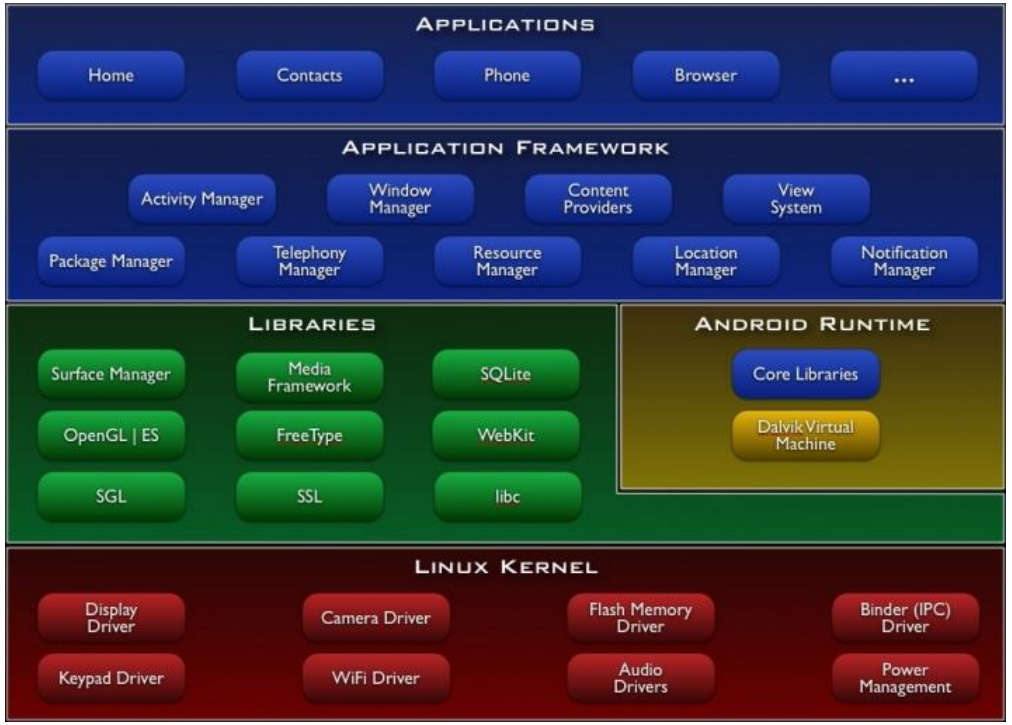
안드로이드(1/2)

안드로이드(Android)

- OS, Middleware, Key Application을 포함하는 모바일 기기의 S/W 집합체

안드로이드 구조

- Linux Kernel
  - 하드웨어의 관리 및 하드웨어와 OS 사이의 인터페이스 제공
- Libraries & Android Runtime
  - 여러 기능을 사용할 수 있는 라이브러리와 가상 머신 및 컴파일러
- Application Framework
  - 개발자들이 다양한 서비스를 사용할 수 있도록 하는 인터페이스 제공
- Application
  - 실제로 사용자들이 사용하는 어플리케이션



안드로이드 구조도

안드로이드(2/2)

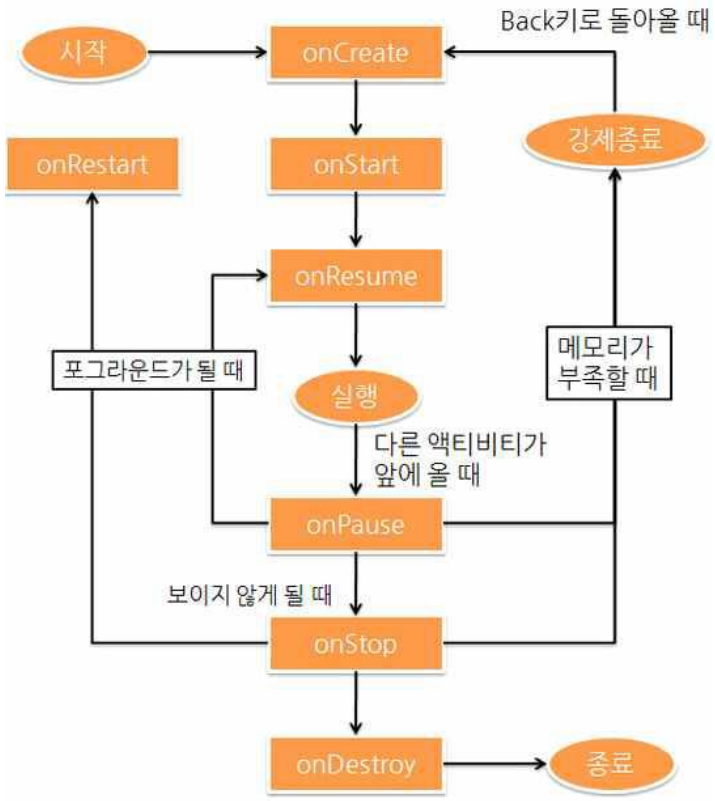
안드로이드 어플리케이션

- Activity, Intent, Service, Content Provider 로 구성
- 컴포넌트들의 목록을 AndroidManifest.xml 에 기록
- xml: 선언된 컴포넌트의 기능과 요구사항을 기록하여 저장하는 파일
- Activity
  - 어플리케이션에서 하나의 화면을 지칭
  - Activity base class를 상속하여 구현한 하나의 클래스
  - 사용자에게 view와 event 응답을 제공하는 인터페이스
- Intent
  - 화면과 화면 사이를 이동할 때 사용하는 클래스
  - Intent가 어떤 작업에 대한 요청
  - 어플리케이션이 원하는 것에 대한 정보를 담고 있다.
- Service
  - UI와 상관없이 아주 오랫동안 존재하며 실행되는 코드
    예) 재생목록에서 노래를 재생하는 미디어 플레이어
- Content Provider
  - 자신의 데이터를 SQLite 데이터베이스 또는 다른 방법으로 파일에 저장할 수 있다.
  - 어플리케이션의 데이터가 다른 어플리케이션과 공유할 필요가 있을 때 유용
  - 다른 어플리케이션이 데이터를 저장하거나, 가져오는 작업을 할 수 있도록 한다.

어플리케이션 구조(1/6)

Android Activity

- 안드로이드 디바이스에서 화면에 표시되는 UI 구성 요소의 집합
- 하나의 화면에는 하나의 Activity만 실행 가능
- 각 요소를 객체를 생성하여 원하는 동작을 구현 가능
- Java, Kotlin 으로 구현



Activity의 생명 주기

어플리케이션 구조(2/6)

Android XML

- 사용자에게 보여지는 레이아웃을 작성하는 파일

Android Manifest

- 어플리케이션의 권한을 설정하는 파일
- Activity, library, version 에 대한 정보를 명시할 수 있다.

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.IoTPractice">

    <uses-permission android:name="android.permission.BLUETOOTH" />
    <uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
    <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
```

Manifest 의 일부

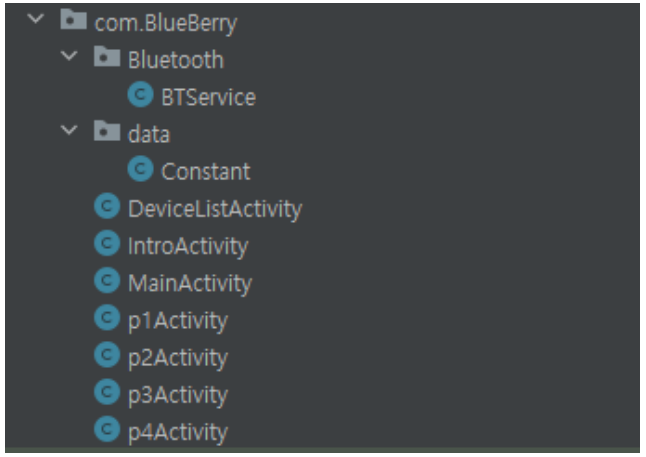
```xml
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@drawable/bg" >

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@drawable/bg_title"
        android:gravity="center_vertical" >
        <!--
        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="@string/app_name"/>  -->

        <ImageView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:src="@drawable/new_title"
            android:contentDescription="임베디드시스템 블루투스"/>

        <Button
            android:id="@+id/btBTN"
            android:layout_width="50dp"
            android:layout_height="wrap_content"
            android:layout_marginRight="20dp"
            android:background="@drawable/selector_bt_btn"
            android:onClick="clickFunc"
            android:visibility="gone"/>
```

MainActivity의 XML

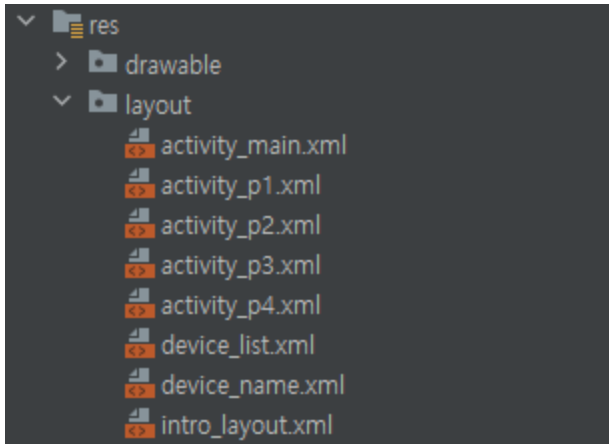
어플리케이션 구조(3/6)

BlueBerry Application

- 라즈베리 파이에 연결된 센서 및 장치를 제어하기 위한 안드로이드 어플리케이션
- Activity와 Layout으로 구성
- IntroActivity, MainActivity 그리고 과제 수행을 위한 Activity
- 각 Activity을 화면상에 표현하는 xml 파일



Activity List



XML List

어플리케이션 구조(4/6)

MainActivity & activity_main.xml

- xml에서는 버튼, 토글 버튼, 텍스트 뷰 등의 요소를 추가하여 사용자 인터페이스를 구성

버튼 – activity_main.xml

```
<Button
    android:id="@+id/red_btn"
    android:layout_width="0dp"
    android:layout_weight="1"
    android:layout_height="wrap_content"
    android:text="@string/red_led"
    android:textStyle="bold"
    android:onClick="clickFunc"/>
```

- id는 해당 객체의 고유한 이름을 나타내며, onClick은 해당 버튼을 눌렀을 때의 동작을 나타낸다.
- width/weight/height 등의 요소는 객체의 모양과 위치를 결정하는 요소이다.

어플리케이션 구조(5/6)

MainActivity & activity_main.xml

- xml에서 함수를 호출하면 해당 xml과 연결된 activity에서 해당 함수를 실행

clickFunc() – MainActivity

```
public void clickFunc(View v) {
        int id = v.getId();
        if (id == R.id.btBTN) {
            if (mBTService != null) mBTService.stop();
            startActivityForResult(new Intent(this, DeviceListActivity.class), REQUEST_CONNECT_DEVICE);

        } else if (id == R.id.tg_btn_buzzer) {
            sendMessage(((ToggleButton) v).isChecked() ? "@BUZ,1#" : "@BUZ,0#");

        } else if (id == R.id.red_btn) {
            ((Button) v).setTextColor(!bRedOn ? Color.RED : Color.BLACK);
            sendMessage(!bRedOn ? "@RED,1#" : "@RED,0#");
            bRedOn = !bRedOn;
}
```

- 눌려진 버튼의 id를 바탕으로 적절한 기능을 수행한다.

어플리케이션 구조(6/6)

MainActivity & activity_main.xml

- initUI()는 xml에 있는 요소들을 MainActivity에서 초기화한다. findViewById() 함수를 통해 연결된다.

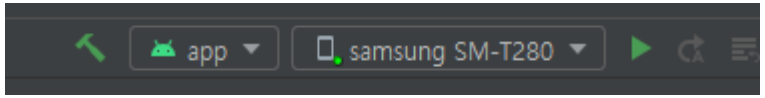
initUI() – MainActivity

```
private void initUI(){
    mAppRes      = this.getResources();
    mBtBTN = (Button)findViewById(R.id.btBTN);
    mBtBTN.setVisibility(View.VISIBLE);
    mBtBTN.setBackgroundResource(R.drawable.selector_bt_btn);
    mBtBTN.setSelected(false);
    mTvTemp     = findViewById(R.id.tv_temp);
    mTvIll     = findViewById(R.id.tv_ill);
    mRedBtn        = findViewById(R.id.red_btn);
    mGreenBtn   = findViewById(R.id.green_btn);
    mBlueBtn    = findViewById(R.id.blue_btn);
    mBTService = new BTService(this, mHandler);
}
```

안드로이드 스튜디오 설정

안드로이드 - 갤럭시 탭 설정

- e-루리에서 'BlueBerry' 어플리케이션 소스 파일을 다운받아 안드로이드 스튜디오에서 실행한다.
- 갤럭시 탭을 케이블을 이용하여 PC에 연결한다.
- 갤럭시 탭 [설정] - [디바이스 정보] - [소프트웨어 정보] - [빌드 번호] 를 여러 번 클릭한다.
- [시스템] - [개발자 옵션] - [USB 디버깅] 이 활성화 되어있는지 확인한다.
- 연결 과정에 권한 승인 요청 메시지가 출력되면 승인한다.
- 안드로이드 스튜디오 오른쪽 상단에 연결 여부를 확인한다.



- 기존에 설치한 '임베디드 시스템 블루투스' 어플리케이션을 삭제한다.
- 안드로이드 스튜디오 오른쪽 상단의 화살표를 눌러 빌드하여 설치한다.

이미지 창 추가하기(1/4)

p1Activity 의 구조

- p1Activity는 [Capture] 버튼만 구현되어 있다. 해당 버튼을 누르면 다음과 같은 함수가 동작한다.

```
public void clickFunc(View v) {
        int id = v.getId();
        if (id == R.id.btBTN) {
            if (mBTService != null) mBTService.stop();
            startActivityForResult(new Intent(this, DeviceListActivity.class), REQUEST_CONNECT_DEVICE);
        } else if (id == R.id.p1_capture) {
            sendMessage("@SHT#");
        }
    }
```

- (핵심) capture 버튼을 누르면 “@SHT#” 메시지가 라즈베리 파이로 전달된다.

이미지 창 추가하기(2/4)

BTServer.c

- 수신한 데이터를 처리하는 parserCmd() 함수에는 “@SHT#” 를 처리하는 구문이 있다.

```
if(!strncmp(&rx_buf[index], "SHT", 3)){
        index += 3;
        if(rx_buf[index] == '#'){
                printf("capture image₩n");
                CaptureImage();
                printf("Send image₩n");
                sendImageData();
        }
```

- CaptureImage(): 카메라를 이용하여 사진을 촬영하는 함수
  - 시스템 명령어를 사용
- SendImageData(): 설정한 경로에 있는 이미지를 전송하는 함수
- (핵심) 전송하는 메시지의 구조를 파악하여 해당 메시지를 읽어 올 수 있도록 설정

이미지 창 추가하기(3/4)

activity_p1.xml

- p1Activity의 구성요소들을 정의한 레이아웃 파일
- 전송받은 이미지를 출력할 공간이 필요
- 버튼 아래(line 46)에 다음 구문을 추가한다.

```
<ImageView
    android:id="@+id/p1_imageView"
    android:layout_width="match_parent"
    android:layout_height="350dp"
    android:layout_marginStart="20dp"
    android:layout_marginLeft="20dp"
    android:layout_marginTop="20dp"
    android:layout_marginEnd="20dp"
    android:layout_marginRight="20dp"
    android:layout_marginBottom="20dp" />
</LinearLayout>
```

- 해당 이미지 뷰어의 id는 ‘p1_imageView’ 이다.
  해당 구성요소를 p1Activity에 정의하여 전송 받은 데이터를 출력한다.

이미지 창 추가하기(4/4)

p1Activity

- ClickFunc() 함수에 다음 구문을 입력한다.

```
public void clickFunc(View v) {
    switch(v.getId()) {
        case R.id.btBTN:
            if (mBTService != null) mBTService.stop();
            mPhoto.setVisibility(View.GONE);
            startActivityForResult(new Intent( packageContext: this,
                                com.BlueBerry.DeviceListActivity.class),
                        REQUEST_CONNECT_DEVICE);
            break;
        case R.id.p1_capture:
            sendMessage("@SHT#");
            break;
    }
}
```

- line 45~50 부근에 이미지 뷰어를 선언한다.

  import android.widget.ImageView; (클래스 호출)

  private ImageView mPhoto; (변수 선언)

- initIU() 함수에 선언한 이미지 뷰어를 xml 파일의 요소와 연결하고 초기화 한다. 해당 구문을 추가한다.

  mPhoto = findViewById(R.id.p1_imageView)

- HandleMessage() 함수의 switch 구문의 3번째, Constant.MESSAGE_READ 항목으로 이동한다.

- 해당 위치에 다음과 같은 구문을 입력한다.

```
case Constant.MESSAGE_READ:
    byte[] readBuf = (byte[]) msg.obj;

    if(msg.arg2 == 1) { // Image
        int offsetHead = 4;
        BitmapFactory.Options options = new BitmapFactory.Options();
        options.inSampleSize = 1;
        Bitmap bm = BitmapFactory.decodeByteArray(readBuf, offsetHead,
                length: msg.arg1 - offsetHead, options);
        mPhoto.setImageBitmap(bm);
        mPhoto.setVisibility(View.VISIBLE);
        mDialog.dismiss();
    } else {
        String readMessage = new String(readBuf, offset: 0, msg.arg1);

        Log.d(TAG, msg: "readMessage : "+ readMessage);
        setResultValue(readMessage);
```

새로운 페이지 만들기(1/2)

Activity

- p2Activity에 p1Activity의 내용을 복사한다.
- p1Activity로 정의되어 있는 항목을 모두 p2Activity로 변경한다.
  1) 클래스 이름, 2) onCreate(), 3) Handler에 해당 항목이 있다. **사진과 관련된 변수 제거
- onCreate 함수 내에서 레이아웃을 activity_p2 로 변경한다.

```java
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_p1);

    mBluetoothAdapter = BluetoothAdapter.getDefaultAdapter();

    if (mBluetoothAdapter == null) {
        Toast.makeText( context: this,  text: "Bluetooth is not available", Toast.LENGTH_LONG).show();
        finish();
        return;
    }
}
```

```xml
    <activity
        android:name="com.BlueBerry.DeviceListActivity"
        android:label="select a device to connect"
        android:screenOrientation="portrait"
        android:theme="@android:style/Theme.Dialog" />
    <activity android:name="com.BlueBerry.MainActivity"/>
    <activity android:name="com.BlueBerry.p1Activity"/>
    <activity android:name="com.BlueBerry.p2Activity"/>
    <activity android:name="com.BlueBerry.p3Activity"/>
    <activity android:name="com.BlueBerry.p4Activity"/>
```

- AndroidManifest.xml 항목에서 해당 activity를 추가한다.
  (현재는 설정이 되어 있다.)

새로운 페이지 만들기(2/2)

XML

- activity_p2에 activity_p1 의 내용을 복사한다.
- 블루투스 연결 버튼을 제외한 부분을 지우고 사용한다.
- p1과 관련된 부분을 찾아 p2로 수정한다.
- 데이터를 받아오는 창과 버튼은 activity_main.xml을 참고하여 구현한다.

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".p1Activity">
    <TextView
        android:id="@+id/practice1_banner"
        android:layout_width="match_parent"
        android:layout_height="100dp"
        android:text="실습 1번"
        android:background="#22FFB2"
        android:gravity="center_horizontal|center_vertical"
        android:textSize="50sp"
        android:layout_marginTop="10dp"
        android:layout_gravity="top"
        android:textStyle="bold">
    </TextView>
    <Button
        android:id="@+id/btBTN"
        android:layout_width="60dp"
        android:layout_height="60dp"
        android:layout_gravity="center_horizontal|center_vertical"
        android:layout_margin="20dp"
        android:background="@drawable/selector_bt_btn"
        android:onClick="clickFunc">
```

고생하셨습니다

Thank you!